

ZeroAds Downloader

Unified Ad-Free Social Media Video & Audio Downloader - Architecture Design Proposal

PREPARED FOR	AUTHOR	DATE / VERSION
Development Phase 1	Engineering Team Proposal	July 2026 / v1.0

1. Executive Summary

This document outlines the product requirements and technical architecture for **ZeroAds Downloader**. The proposed mobile application is an ad-free, high-performance media utility designed to extract, download, and store high-quality video and audio files from multiple social media platforms (including YouTube, TikTok, Facebook, Snapchat, X, Instagram, etc.) directly into a mobile device's local storage.

A critical differentiator of this utility is the **Smart Queue Downloader**. In contrast to existing alternatives that drop failed downloads during network changes, this application integrates a native-level queue management system. It automatically pauses active operations during disconnects and resumes progress immediately upon connection restoration, running entirely background-independent of UI lifecycle states.

2. Core Functional Requirements

- **Universal Media Extraction:** A single-input field that extracts media direct download links from dynamic, authenticated, or obfuscated URL configurations.
- **Zero Advertising (Ad-Free UX):** An interface designed strictly around clean UI/UX paradigms, void of banner ads, popups, redirects, or visual clutter.
- **Audio vs. Video Toggle:** Automatic multi-stream extraction allowing users to select quality levels (e.g., 1080p, 720p) or convert direct media streams to standalone high-bitrate audio formats (MP3/M4A).
- **Native Share-Target Integration:** Mobile operating system intent registration so users can hit the "Share" button inside apps like Instagram or TikTok and directly open ZeroAds Downloader with the URL automatically populated.
- **Background Resilient Transfer Engine:** Auto-pause on cellular dropouts or complete disconnection, with automated resume once Wi-Fi or active mobile data paths are restored.

App Store Distribution & Policy Compliance Warning

Google Play Store policies strictly prohibit downloading content from YouTube (ToS violations). Publishing an app that downloads YouTube media will result in immediate store rejection or account suspension. **Strategy:** Distribute the complete build (including YouTube) as an APK through an independent website, F-Droid, or GitHub. Optionally, build a compliant, separate "Light" version for the Play Store that excludes YouTube domain extraction.

3. Architecture Design

Because client-side scraping of highly dynamic platforms (like TikTok, Instagram, and YouTube) is extremely fragile due to constant algorithm modifications, a **hybrid architecture** is specified. This delegates media extraction to a scalable cloud backend while relying on native system services to handle file transfers.

System Topology Workflow

1. **User App** detects user-shared link → 2. Sends URL payload to **API Backend** → 3. Backend spins up a parser instance (utilizing a live-updated, microservices-wrapped yt-dlp core) → 4. Backend returns clean, direct CDN download URLs (video and audio paths) to **App** → 5. App schedules native transfer queues via the operating system's direct **Download Manager API**.

3.1 Frontend App Architecture (React Native / Flutter / Native)

The client-side mobile application behaves as a clean input shell. It contains the clipboard monitoring services, network monitors, share intents, and manages the queue visualization.

- **Share Intent Module:** Integrates library bindings (e.g., Expo Share Intent or react-native-share-menu) to map shared payloads directly to the download interface.
- **System Network Listeners:** Leverages active network status observers to transition visual UI queues from "Downloading" to "Paused - Network Interrupted" dynamically.

3.2 Backend Service (Extraction API)

The backend service acts as a translation layer. It receives the target social link, parses it securely, and delivers static download links to the app.

Component	Technology Recommended	Primary Responsibility
API Gateway	FastAPI (Python) / Node.js	Receives HTTP requests, executes rate-limiting, and triggers parser logic.
Parser Core	yt-dlp (Python Library Wrapper)	Dynamically updates daily to circumvent social platform extraction blocks.
Cache Store	Redis	Caches generated CDN download URLs for 15 minutes to reduce server resource abuse.

4. Implementation Strategy: Auto-Resume & Background Downloading

For background resilience, the application should **never** use raw web fetch requests (which immediately drop when the app is minimized). Instead, it must hand off the URLs to the mobile operating system's native download pipelines:

4.1 Android Implementation (Native DownloadManager API)

The Android platform provides an enterprise-grade system service called `DownloadManager`. It runs completely independent of your app's lifecycle, managing notifications and retrying automatically.

```
// Android native-side queue registration via Java/Kotlin:
DownloadManager.Request request = new DownloadManager.Request(Uri.parse(backendExtractedUrl));

request.setTitle("Downloading Video");
request.setDescription("ZeroAds Background Transfer Engine");
request.setDestinationInExternalPublicDir(Environment.DIRECTORY_DOWNLOADS, "ZeroAds_Video.mp4");

// Force native download engine to pause when connection drops and wait for reconnect
request.setAllowedNetworkTypes(DownloadManager.Request.NETWORK_WIFI | DownloadManager.Request.NETWORK_MOBILE);
request.setRequiresDeviceIdle(false);
request.setNotificationVisibility(DownloadManager.Request.VISIBILITY_VISIBLE_NOTIFY_COMPLETED);

DownloadManager manager = (DownloadManager) getSystemService(Context.DOWNLOAD_SERVICE);
long downloadId = manager.enqueue(request); // Handoff complete. OS manages pause/resume.
```

4.2 iOS Implementation (URLSession Background Configuration)

On iOS, transfers are handled similarly via a background-configured `URLSession` session, passing execution tasks directly over to the system kernel:

```
// iOS background session setup
let config = URLSessionConfiguration.background(withIdentifier: "com.zeroads.downloader.background")
config.waitsForConnectivity = true // Key setting: holds session open when internet is lost
config.discretionary = false

let session = URLSession(configuration: config, delegate: self, delegateQueue: nil)
let downloadTask = session.downloadTask(with: URL(string: backendExtractedUrl)!)
downloadTask.resume()
```

5. Development Roadmap

- Phase 1 (Proof of Concept):** Develop a basic Python FastAPI backend wrapping `yt-dlp`. Construct a basic React Native app UI that receives shared links and fires them to the local backend, returning a single unobfuscated video URL to test downloading.
- Phase 2 (Production Handoff):** Integrate native Android `DownloadManager` and iOS `URLSession` modules. Implement clipboard listener routines and OS-level share intents.
- Phase 3 (Platform Expansion & Edge-Cases):** Implement advanced decryption strategies for Instagram/TikTok watermarked video downloads, private-account handling, and dynamic video resolution selector panels.
- Phase 4 (Deployment):** Host the extraction API on auto-scaling containers (such as AWS ECS or Google Cloud Run) and launch APK distribution structures for Android users, bypass-install targets, or an Apple IPA deployment system.